

M2 S2I Session 1 – Examen IT4-Blockchain

Nicolas Herbaut

UFR27, Université Paris 1 Panthéon-Sorbonne 2022-2023

1 Questions sur les concepts de la blockchain (10 pts)

1.1 Blockchain et technologie

1.1.1 Double Paiement (2pt)

Expliquez comment Nakamoto prévoit de résoudre le problème de double paiement.

1.1.2 Valideur Ethereum PoS (2pt)

Expliquez le rôle des validateurs pour le consensus Proof-of-Stack d'Ethereum. Comment s'assurer que les validateurs soient honnêtes?

1.1.3 DAO (2pt)

Que signifie DAO dans le cadre de la blockchain? Expliquez les avantages par rapport à une entreprise classique.

1.2 Blockchain et aspects juridiques

1.2.1 BC + Protection des données (2pt)

Pourquoi les caractéristiques inviolables (tamper-proof) de la blockchain entrent-elles en conflit avec les exigences de protection des données ?

1.2.2 Smart Contracts (2 pt)

Les contrats intelligents sont-ils juridiquement contraignants ? Donnez au moins une raison.

2 Patterns Blockchain (10 pt)

2.1 Blockchain en Entreprise de services du numérique

Les ESN (Entreprise de services du numérique) ont souvent recours à l'externalisation des prestations qu'elles vendent à leur client. L'avantage étant que l'utilisation de ressources externalisées permet d'augmenter la marge de ces entreprises sans pour autant augmenter la masse salariale.

Les prestataires externes peuvent être soit des personnes à leur compte (freelance) soient des personnes en contrat avec une autre ESN.

Les ESN doivent s'assurer que leur recrutement est de qualité, que les intervenants externes ont toutes les qualifications requises et que les projets sur lesquels ils sont intervenus précédemment ont été des succès.

Les ESN sont regroupées au sein de la Fédération Syntec.

2.2 Questions

2.2.1 Avantages d'une solution blockchain

En tant que consultant blockchain, vous montez dans le même ascenseur que Laurent Giovachini (Ingénieur X, DG de Sopra Steria et président de la fédération Syntec) dans une tour à la Défense. Expliquez-lui en 3 minutes comment le déploiement d'une blockchain commune à toutes les ESN françaises leur permettrait d'optimiser l'utilisation de leurs ressources externes.

2.2.2 Diagramme de solution

Votre "elevator speech" a porté ses fruits: vous avez maintenant rendez-vous avec Mohammed Sijelmassi (Maitrise MIAGE et CTO de Sopra Steria) pour lui expliquer comment implémenter la solution que vous avez décrite à Laurent Giovachini.

À l'aide d'un diagramme de composants, montrer comment déployer la solution à l'aide de recettes déjà éprouvées (software patterns). Précisez quelle technologie blockchain pourrait être utilisée.

Vous pouvez ajouter une description textuelle de votre réponse.

2.2.3 Lister les patterns pertinents

Donnez au moins 3 patterns les plus pertinents pouvant être utilisés lors de l'implémentation de votre solution. Pour chaque pattern, expliquez comment et pourquoi il est utilisé. Vous justifierez votre réponse en utilisant les propriétés des patterns par rapport au cas d'utilisation.

1. Interaction with External

Ces patterns résolvent les problèmes liés à l'interaction entre les applications basées sur la BC et les systèmes externes, tels que la récupération de data externes fiables, l'intégration avec des systèmes existants et la gestion des événements.

Oracle Pattern : permet à une application BC d'interagir avec des sources de data externes.(météo, valeur USD/ETH)
L'oracle possède une clé privée pour publier des infos du monde extérieur vers la bc

Reverse Oracle : permet aux composants externes d'un système existant de demander des data avec une blockchain en utilisant des SC.
Rendre disponible des infos de la bc aux composants externes.

Legal & SC pair : résout le problème de l'exécution sécurisée des accords numériques en fournissant une paire de contrats, un contrat légal et un contrat intelligent, qui travaillent ensemble pour exécuter les conditions d'un accord numérique, ce qui permet une exécution fiable et transparente des accords.

Doc contractuel qui a des conditions => il aurait une image sous forme de smart contract dans la bc. Les 2 contrats possèdent des liens vers l'autre version du contrat il peut y avoir des conditions hors chaîne (ex: j'ai payé le contrat)

Ex: dans le domaine de l'immobilier, où un contrat légal peut être utilisé pour définir les termes et conditions d'un accord de location, tandis qu'un contrat intelligent peut être utilisé pour exécuter automatiquement les paiements de loyer et les dépôts de garantie en fonction des conditions définies dans le contrat légal.

2. Data Management

Ces patterns résolvent les problèmes liés à la gestion des data dans les applications basées sur la BC, tels que le stockage de l'état interne pour faciliter les transactions futures, la représentation numérique d'actifs physiques ou virtuels et le contrôle de l'accès aux data stockées sur la BC.

Encrypting On-Chain Data : Assurer la confidentialité des data stockées sur la BC en les chiffrant.
Les gens s'envoient une clé de déchiffrement symétriques entre eux pour lire les données.

Tokenisation : permet la représentation numérique d'actifs physiques ou virtuels sur la BC (colored coin).
Prouve que l'on est en possession d'un bien en particulier dans la vie réelle. Utile pour la distribution.

Off-chain data storage : Utiliser le hachage pour garantir l'intégrité d'ensembles de data arbitrairement grands qui ne peuvent pas être placés directement sur la BC, on stocke la BDD sur un cloud et on met le hash de cette BDD sur une BC par ex.
On stock le hash de la donnée dans la bc (qte stocké très faible) et ailleurs. Si on signe un contrat, on calcul le hash d'un contrat et on stock le hash dans la bc (que l'on aura copié plusieurs fois), puis pr vérifier que le document n'a pas été altéré, il suffit de vérifier que son hash correspond tjrs à celui présent dans la bc

Note : objet final, attribut pas final. L'obj supprime les données et le code quand on maj le contrat

Embedded permission : résout le problème de la gestion des autorisations d'accès aux données dans un smart contract en incorporant les règles d'autorisation directement dans le code du contrat, ce qui permet de contrôler l'accès aux données stockées sur la blockchain (on vérifie avec la clé publique de la personne).
Avant l'exécution de certaines fonction, on vérifie l'identité des personnes qui les exécutent
On vérifie si la clé publique correspond aux personnes qui ont le droit d'interagir avec telle ou telle fonction du smart contract

Data contract : Stocker les data dans un SC distinct. Si on l'utilise avec Contract registry, on peut avoir la maj du smart contract et la préservation des données sans toucher à notre code.

Factory contract : permet de créer des instances de contrats intelligents avec des paramètres personnalisés. On est sûr que le factory contract ne changera jamais donc c'est très sécurisé et c'est simple à utiliser. Par contre c'est un peu plus cher que de déployer (coût de la factory + créer un contrat à partir de la factory au lieu de créer à partir du source code)
Ex: Dans une application de soins de santé basée sur la blockchain, ce pattern a été utilisé pour créer des contrats intelligents personnalisés pour chaque patient, permettant ainsi une gestion efficace des dossiers médicaux et une meilleure coordination entre les professionnels de la santé.

Incentive execution : résout le problème de la motivation des utilisateurs à exécuter des fonctions de contrat intelligent en offrant une récompense pour l'invocation de l'exécution, ce qui encourage les utilisateurs à participer activement à l'application blockchain.
Une personne a qui ont demandé de créer une fonction sur un smart contract, normalement ça lui coûte de l'argent.
Le but est qu'il soit dédommagé après avoir appelé une fonction.
Il appelle une fonction qui va lui donner le droit d'appeler une autre fonction qui va lui donner des codes.

Ex: une application de vote en ligne où les électeurs peuvent soumettre leur vote en utilisant un hash créé hors de la chaîne pour lier leur autorité à leur vote, ce qui permettrait aux électeurs de voter de manière anonyme tout en garantissant que chaque vote est authentique et autorisé.

State channel : résout le problème de la scalabilité des transactions sur une blockchain en permettant aux participants de maintenir les états intermédiaires des micro-paiements hors de la chaîne, ne stockant que les paiements finalisés
Ex: une application de jeux en ligne où les joueurs peuvent effectuer des micro-transactions pour acheter des objets virtuels. Les transactions seraient effectuées hors de la chaîne, mais les soldes des joueurs seraient mis à jour sur la blockchain uniquement lorsqu'ils quittent le jeu ou effectuent un achat important.
Pr le cas du wifi d'un hotel facturé au Mo : au lieu d'envoyer plsr transactions, on maj la transaction au fur et à mesure avant d'envoyer sur la bc. alternative : contract sur lequel tt le monde peut interagir (mais cela coûte de l'argent à chaque changement).Pb: il faut avoir confiance en le canal de communication avant que la donnée soit envoyée

3. Security

Ces patterns résolvent les problèmes liés à la sécurité dans les applications basées sur la BC, tels que la validation des transactions, la gestion des identités, la protection de la vie privée et le suivi de l'historique des transactions.

Multiple authorization : résout le problème de la sécurité des mises à jour sur une blockchain en exigeant une autorisation de plusieurs adresses avant qu'une mise à jour puisse être effectuée, ce qui garantit que les mises à jour ne peuvent être effectuées que par des parties autorisées.
Executer cette opération si un certain nb de personnes sont d'accord parmis toutes celles qui sont présentes.

Off-chain secret enabled dynamic authorization : Utilisation d'un hachage créé en dehors de la chaîne pour lier dynamiquement l'autorité d'une transaction.
Ex: app de vote en ligne où les électeurs peuvent soumettre leur vote en utilisant un hash créé hors de la chaîne pour lier leur autorité à leur vote, ce qui permettrait aux électeurs de voter de manière anonyme tout en garantissant que chaque vote est authentique et autorisé.
Qqun crée des contrats avec un verrou qui ne peut être ouvert que grâce aux clés que l'autre possède. Mais au moment du verrou il n'a pas défini qui aura la clé.
En gros : on a pas besoin de savoir à qui sera donné la clé au moment de la création du smart contract. La clé ne peut être utilisée qu'une seule fois

X-confirmation : Attente d'un nombre suffisant de blocs en guise de confirmation pour garantir qu'une transaction ajoutée à la BC est immuable avec une forte probabilité.

4. Structural Patterns of Contract

Contract registry : résout le problème de la mise à jour des smart contracts sur une blockchain en permettant de localiser l'adresse de la dernière version d'un smart contract en cherchant son nom dans un registre de contrats avant de l'invoquer.
Ex: une application de marché où les vendeurs peuvent déployer des contrats intelligents pour vendre leurs produits et les acheteurs peuvent invoquer ces contrats en utilisant le nom du contrat pour acheter les produits.